Lecture 34: Vision-Language Models II

How a model draws

The same model wrote the caption and drew the picture

You ask a chat model to describe a photo. Then you ask it to draw one. Same conversation, same model, no tool call, and you can ask it to change the third object from the left.

That should strike you as architecturally strange, because of what you built in ch10.

Your diffusion model works in **continuous** space. It predicts a real valued noise vector and refines a real valued image over 50 passes.

A language model emits **discrete** tokens, one at a time, each an index into a fixed vocabulary, each sampled from a softmax.

These are not the same kind of object. Something has to give.

Three ways to resolve it, and nobody agrees

L33 said reading an image is settled. Drawing one is not. There are three live answers, all shipping in real systems today:

Make the image discrete	Chop it into a vocabulary and predict tokens. One model, one loss, no diffusion at all.
Make the model do both	Autoregressive on text, diffusion on images, one shared transformer.
Use different encoders	One representation for understanding, another for generating, sharing only the trunk.

We take the first one seriously enough to **measure it on our own letters**, because it is the one with an obvious cost - and the cost turns out to be exactly the fragility ch10 ran into.

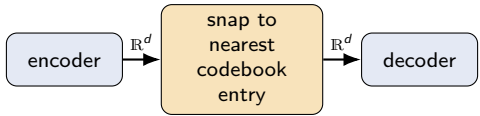
Outline

A picture made of words

If an image were a sequence of symbols from a fixed vocabulary, GPT would already know what to do with it.

VQ-VAE - an autoencoder with a dictionary

Start from the autoencoder of ch8, and change exactly one thing.



the latent is now an **integer**

Keep a learned table of K vectors - the **codebook**. After the encoder produces a latent vector, replace it with the nearest entry in that table and pass the entry to the decoder. **Vector Quantized Variational Autoencoder** (**VQ-VAE**), van den Oord et al. (2017).

The latent is no longer a point in a continuous space. It is **the index of a table row** - a symbol, exactly like a word.

The gradient problem, and the shameless fix

"Pick the nearest entry" is an $\arg\min$. Its derivative is zero almost everywhere and undefined at the boundaries, so gradients cannot flow back to the encoder and the encoder never learns.

The straight-through estimator. On the way forward, use the codebook entry. On the way back, **pretend the quantizer was not there** and copy the decoder's gradient straight onto the encoder's output.

It is not the true gradient of anything. It is a deliberate lie that points in approximately the right direction, and it works well enough that the whole field is built on it.

Two extra loss terms then pull the codebook entries and the encoder outputs toward each other, so the lie stays small.

A recurring pattern in deep learning: when a discrete step blocks the gradient, invent a differentiable fiction and check empirically that it trains.

The codebook is a vocabulary

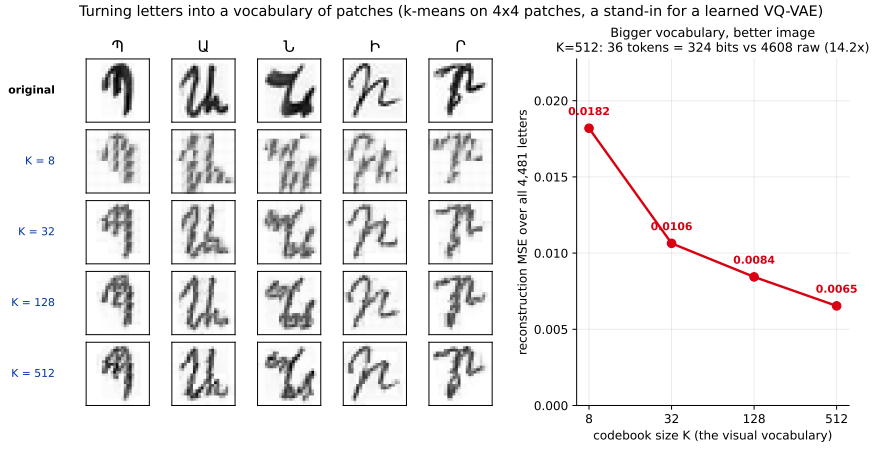
Once an image is a grid of integers, the analogy to text stops being an analogy.

Chameleon (Meta, 2024)	**Text**	**Image**
Vocabulary size	65,536	8,192
Tokens per item	varies	1,024 per 512×512 image
Produced by	byte-pair encoding	learned image tokenizer
Predicted by	one softmax	**the same softmax**

There is now **one sequence**, containing both kinds of token, predicted by **one model** with **one loss**. Nothing in the transformer needs to know which tokens were pixels. This is the cleanest answer to the opening contradiction - and it is clean precisely because it forces images to become discrete.

So what does forcing them to be discrete cost?

Measured on the ch10 dataset: 4,481 handwritten letters at 24×24, cut into 4×4 patches, with a K-entry codebook fitted by k-means.



k-means stands in for a learned VQ-VAE encoder, which would do better - so read this as a **lower bound** on quality.

Two things to read off that figure

1. Sharply diminishing returns.

K=8 → 32 cuts the error by **41%** (0.0182 → 0.0106). K=128 → 512 buys only **22%** for four times the vocabulary.

Growing K is not a solution, which is why the field went to cleverer quantization instead - MAGVIT-2 reaches a vocabulary of 2^{18} only by dropping the nearest-neighbour lookup altogether.

2. Even at K=512 the strokes break up.

Look at the letters, not the number. The reconstruction is speckled and the strokes come apart at patch boundaries.

Each patch is quantized **independently**, with no knowledge of its neighbours. A stroke crossing a boundary has no mechanism holding it continuous.

This is ch10's finding again. These strokes are 1-2 pixels wide. There, halving the image twice erased them before the deep layers saw them. Here, quantizing 4×4 blocks independently breaks them. Thin high-frequency structure is what every compression scheme destroys first - and text in an image is exactly that.

Compression, stated as a number

With $K = 512$ and 4×4 patches, one 24×24 letter becomes:

$$36 \text{ tokens} \times \log_2 512 \text{ bits} = \textbf{324 bits}, \text{ against } 24 \times 24 \times 8 = \textbf{4,608 bits} \text{ raw.}$$

$$\text{A } \textbf{14.2}\times \text{ compression.}$$

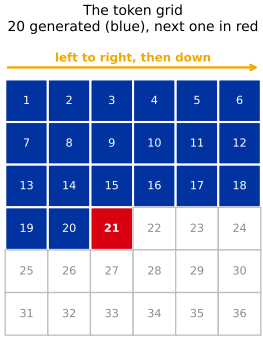
That compression is the *point*. A sequence model cannot afford one token per pixel - $512 \times 512 \times 3$ would be 786,432 tokens.

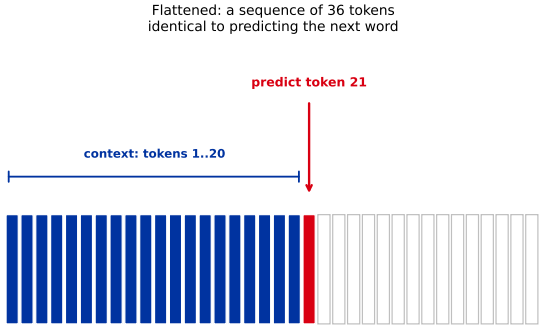
Chameleon's tokenizer gets 512×512 down to **1,024** tokens. That is the only reason putting an image in a language model's sequence is possible at all.

Next-token prediction, on a picture

There is no new algorithm in this section. That is the whole point of it.

Flatten it and predict the next one





Read the token grid top to bottom, left to right. Now it is a sequence, and generating an image is **exactly** the training objective of GPT: predict token $t+1$ given tokens $1 \ldots t$. Prompt it with text tokens first and you have text-to-image, with no new machinery whatsoever.

Why this was long thought a dead end

It is strictly sequential. 1,024 tokens means **1,024 forward passes**, and token 900 cannot start until 899 is done.

Your ch10 sampler needed 50 passes with DDIM, and every pixel moved in parallel on each one. Autoregressive generation looked like the obviously worse deal.

Raster order is also arbitrary. Nothing about a picture says the top-left corner comes "first". A caption has a real order; an image does not.

Diffusion has no such asymmetry, which is part of why it won the 2021-2024 image generation era outright.

What changed: the goal stopped being "best image generator" and became "one model that does everything". Once you want interleaved text and images, editing by conversation, and knowledge shared between the two, the token formulation stops being a handicap and starts being the only thing that composes.

One model for both

Three architectures, three answers to the same contradiction.

Paradigm 1 - fully discrete

Chameleon (Meta, 2024). Quantize images, put both token types in one vocabulary, train one transformer with one cross-entropy loss. Early fusion, as in L33's Design C.

Maximum elegance. Interleaved text and images in any order, one objective, and every trick that works for language models transfers unchanged.

Pays at the tokenizer. Everything the codebook throws away is gone before the transformer starts - the effect we just measured on the letters.

Training mixed modalities was also unstable enough that Chameleon needed specific fixes (query-key normalisation among them) to converge at scale.

Paradigm 2 - hybrid

Transfusion (Meta, 2024). Do not quantize at all. Keep images as **continuous** patch latents, and run **both** objectives in one transformer.

	Text tokens	**Image patches**
Representation	discrete	continuous
Objective	next-token cross-entropy	diffusion denoising (ch10)
Attention	causal	bidirectional within the image
Transformer	**shared, one set of weights**	
Loss	the two, summed	

This is your ch10 model living inside a language model. The DDPM loss you derived in L28 is one of the two terms. No quantization means no codebook damage, at the cost of a genuinely more complicated system.

Paradigm 3 - decoupled encoders

Janus (DeepSeek, 2024). Observe that understanding and generating want **different** representations, and stop forcing one to serve both.

- To **understand**: SigLIP features (L33) - semantic, abstract, lossy about exact pixels. Good, because you want the meaning.
- To **generate**: quantized tokens - concrete, reconstructable. Good, because you have to output actual pixels.

Only the language model trunk is shared.

The insight worth keeping: "unified" does not have to mean "one representation". Forcing a single encoder to be both semantic and reconstructable makes it worse at each.

GPT-4o's native image generation

The most visible system of all, and the one we know least about. What follows is **reconstruction from published work**, not an official architecture.

Reasonably established:

- Native - not a call out to a separate image model.
- Images as **continuous** latent patches, not discrete codes.
- Explicit begin-image and end-image markers in the token stream.
- A combined language-modelling and denoising loss.

Inferred, and should be labelled so:

- Model sizes and layer counts.
- The exact noise schedule.
- Whether generation is row-by-row or whole-image.
- Data mixing ratios.

Be careful how you repeat this. It is widely described online as "autoregressive image generation", which is at best half right - the reconstruction that fits the evidence is Paradigm 2, autoregressive over *text* with *diffusion* on the image latents.

Why writing text inside images suddenly worked

For years, generated images contained text-shaped marks that were not text. Then, quite abruptly, they contained sentences.

A dedicated image model has a text encoder bolted on and no idea how letters are spelled. A unified model has **spelling knowledge in the same weights** that are drawing the picture.

Same reason editing by conversation works: "make the sign say OPEN" needs the model to hold the picture and the instruction in one context, not to re-generate from a new prompt.

This is ch10's closing point, answered. There, each letter of the word was generated **independently**, so the result had no single hand. A unified model does not carry that independence assumption - the whole image is one sequence, conditioned on everything before it, letters included.

What it costs

The honest frame, as in every chapter.

The bill

Tokens. 1,024 per image is cheap for one picture and ruinous for video: one second at 24 frames is roughly **25,000 tokens** before a single word of prompt.

Sequential decoding. Generating 1,024 tokens one at a time is slow in a way that parallel hardware cannot rescue.

Training. Early fusion means no reusing an existing language model. You pay for everything, from scratch.

Still unsolved:

- the tokenizer is a lossy bottleneck nobody is happy with;
- understanding and generation still interfere with each other during training;
- evaluation is weak - there is no agreed measure of "did it follow the instruction";
- fine detail and text remain the first things to break.

Recap

1. A language model emits discrete symbols; ch10's diffusion works in continuous space. Unifying them means resolving that.
2. **VQ-VAE** makes an image discrete: an autoencoder whose latent is snapped to a codebook entry, trained through a deliberate gradient fiction.
3. We measured the cost on our own letters. K=512 gives 14.2× compression, sharply diminishing returns, and **visibly broken strokes** - thin structure is destroyed first, exactly as in ch10.
4. Once discrete, generating an image is next-token prediction in raster order. No new algorithm.
5. Three unified architectures: fully discrete (Chameleon), hybrid autoregressive-plus-diffusion (Transfusion),

Next: you have now seen how a single transformer can read text, read images, and draw them.

What is left is how such a model is actually *trained to be useful* - instruction following, preferences, and reinforcement learning on top of a pretrained model. That is `ml/llm_training/`, with ch11's reinforcement learning as the foundation.

The unifying idea of this whole chapter: **make everything a sequence of vectors, and one architecture will read all of it.**